

Titans — 한국어 의역본

Titans — 한국어의역본

이 문서는 arXiv:2501.00663 「Titans: Learning to Memorize at Test Time」 (Behrouz, Zhong, Mirrokni (Google Research)) 의 한국어 의역본입니다. 스타일이며 공식 번역이 아닙니다. 수식 · 수치 · 그림은 원문 기준이고, 그림은 원저자의 것입니다. 자연스러운 이해를 위해 문장은 의역했습니다.

저자: Ali Behrouz, Peilin Zhong, Vahab Mirrokni (Google Research) · arXiv:2501.00663v1 (2024-12-31)

초록 (Abstract)

지난 십수 년 동안, recurrent 모델과 attention 을 어떻게 하면 효과적으로 활용할 수 있는가에 대한 방대한 연구가 이어져 왔다. recurrent 모델은 데이터를 고정 크기의 메모리 (이른바 hidden state) 로 압축하려 하는 반면, attention 은 전체 context window 를 훑어보며 모든 토큰 사이의 직접적 의존관계를 포착한다. 그러나 이렇게 의존 관계를 더 정확하게 모델링하는 대가는 제곱 (quadratic) 비용이며, 그 때문에 모델이 다룰 수 있는 context 길이가 고정된 범위로 제한된다.

우리는 과거 context 를 기억하도록 학습하는 새로운 **neural long-term memory module** 을 제안한다. 이 메모리는 attention 이 현재 context 에 주의를 기울이면서도 오래된 과거 정보를 함께 활용하도록 돕는다. 이 neural memory 는 학습을 빠르게 병렬화할 수 있으면서도 추론 (inference) 역시 빠르게 유지한다는 장점이 있다. 메모리의 관점에서 보면, attention 은 context 가 제한되어 있는 대신 의존 관계를 정확히 모델링하므로 **short-term memory** 처럼 작동하고, neural memory 는 데이터를 기억하는 능력 덕분에 **long-term 의, 더 지속적인 memory** 로 작동한다. 이 두 모듈을 바탕으로 우리는 **Titans** 라는 새로운 아키텍처 계열을 소개하고, 메모리를 아키텍처에 효과적으로 통합하는 세가지 변형을 제시한다. language modeling, common-sense reasoning, genomics, time series 과제에 대한 실험 결과는 Titans 가 Transformer 및 최신 linear recurrent 모델보다 더 효과적임을 보여 준다. 나아가 Titans 는 2M 이상의 context window 크기까지 확장되면서도, needle-in-haystack 과제에서 baseline 보다 높은 정확도를 유지한다.

1 서론 (Introduction)

“기억의 진정한 기예는 곧 주의의 기예다!” — Samuel Johnson, 1787

순수 attention 기반 아키텍처인 Transformer 는 in-context learning 능력과 대규모 학습 능력 덕분에 sequence modeling 에서 최첨단 모델로 확고히 자리잡았다. Transformer 의 핵심 구성 요소인 attention 모듈은 associative memory 블록처럼 작동한다. 즉 key-value 연관을 저장하도록 학습하고, query(검색 신호) 와 key(context) 사이의 쌍별 유사도를 계산해 이를 되찾아 온다. 설계상 Transformer 의 출력은 오직 현재 context window 안 토큰들의 직접적 의존 관계에만 조건화된다. 그러나 이렇게 의존 관계를 정확히 모델링하는 대가로 context 길이에 대해 **제곱 시간 · 메모리 복잡도**가 든다. language modeling, video understanding, long-term time series forecasting 같은 복잡한 실제과제에서는 context window 가 극단적으로 커질 수 있어, 이런 downstream 과제에서 Transformer 의 적용성이 제약된다.

Transformer 의 확장성 문제를 극복하기 위해, 최근 연구들은 attention 의 softmax 를 kernel 함수로 대체하는 다양한 **linear Transformer** 변형을 설계했고 (§ 2.1 참조), 이로써 메모리 소비를 크게 줄였다. 하지만 효율성과 긴 context 확장성에도 불구하고, linear Transformer 는 Transformer 만큼 경쟁력 있는 성능을 보이지 못한다. kernel trick 이 모델을 linear recurrent network 로 만들어 데이터를 matrix 값 상태로 압축하기 때문이다. 여기에는 모순적인 사실이 있

다. 한편으로 우리는 확장성과 효율성 (선형 대 제공 복잡도) 을 위해 이런 linear 모델을 쓰는데, 그 이점은 아주 긴 context 에서 두드러진다. 다른 한편으로, 아주 긴 context 는 작은 vector 값이나 matrix 값 상태로는 제대로 압축될 수 없다.

효율성 문제를 넘어서, Hopfield Network 부터 LSTM, Transformer 에 이르는 대부분의 기존 아키텍처는 일반화 (generalization), 길이 외삽 (length extrapolation), 추론 (reasoning) 에서 어려움을 겪는다. 이들은 인간 뇌에서 영감을 받았음에도 각각 다음을 놓치고 있다. (1) short-term memory, long-term memory, meta-memory, 현재 context 에 주의 기울이기 등 학습 과정에 필수적인 요소, (2) 이 요소들이 독립적으로 작동할 수 있는 상호연결된 시스템이라는 점, (3) 데이터로부터 능동적으로 학습하고 과거의 추상을 기억하는 능력. 우리는 효과적인 학습 패러다임이라면 인간의 뇌처럼 서로 구별되면서도 연결된 모듈들이 있어야 하고, 각 모듈이 학습 과정의 한 요소를 담당해야 한다고 주장한다.

메모리 관점 (Memory Perspective)

메모리는 근본적인 정신 과정이며 인간 학습에서 떼어 낼 수 없는 요소다. 제대로 기능하는 메모리 시스템이 없다면 인간과 동물은 기본 반사와 정형화된 행동에 갇힐 것이다. 그런 까닭에 메모리는 Hopfield Network, LSTM, Transformer 등 기계 학습의 여러 선구적 연구에 영감을 주었다.

신경심리학 문헌의 통상적 정의를 빌리면, 대부분의 기존 아키텍처는 메모리를 입력이 일으키는 신경 갱신으로 보고, 학습을 어떤 목적 아래 효과적이고 유용한 메모리를 획득하는 과정으로 정의한다. 이 관점에서 RNN 은 vector 값 메모리 모듈 M (hidden state) 을 갖는 모델로 정의된다. 시각 t 에 새 입력 x_t 가 주어지면 모델은 (1) 함수 $f(M_{t-1}, x_t)$ 로 메모리를 갱신하고 (압축을 동반), (2) 함수 $g(M_t, x_t)$ 로 입력에 해당하는 메모리를 되찾는다. 마찬가지로 Transformer 도 커지는 메모리를 가진 아키텍처로 볼 수 있다. key/value 행렬 쌍이 모델의 메모리 역할을 하며, 모델은 (1) key 와 value 를 메모리에 덧붙여 (압축 없이) 갱신하고, (2) query 와 key 의 유사도를 찾아 value 에 가중치를 부여해 출력을 만든다.

이 관점은 기존 패러다임과 그 결정적 차이를 이해하고 더 효과적인 아키텍처를 설계하는 데 도움을 준다. 예를 들어 Transformer 와 linear Transformer 의 주된 차이는 메모리 구조와 메모리 갱신 방식에 있다. linear Transformer 는 과거 데이터를 고정 크기 matrix 값 메모리로 압축하는 반면, Transformer 는 (context 길이 안에서) 모든 과거 데이터를 압축 없이 보관한다. linear Transformer 와 linear RNN (state space model 포함) 모두 메모리 갱신 단계에서 정보를 압축하지만, 결정적 차이는 메모리의 구조에 있다. linear RNN 은 vector 값 메모리를, linear Transformer 는 matrix 값 메모리를 쓴다. 이런 관점은 다음 질문으로 이어진다. (Q1) 좋은 메모리 구조는 무엇인가? (Q2) 적절한 메모리 갱신 방식은 무엇인가? (Q3) 좋은 메모리 검색 과정은 무엇인가?

인간의 기억을 다시 보면, 그것은 단일한 과정도 단일한 기능도 아니다. 기억은 여러 시스템의 연합체이며 short-term, working, long-term memory 가 각기 다른 신경 구조로 다른 기능을 수행하고 저마다 독립적으로 작동할 수 있다. 이 사실은 다음 질문을 던지게 한다. (Q4) 서로 연결된 여러 메모리 모듈을 통합하는 효율적 아키텍처를 어떻게 설계할 것인가. 끝으로, 기억을 저장한다는 것은 과거의 추상을 부호화해 저장하는 신경 과정이다. 매개변수가 데이터를 선형적으로 부호화하는 단일 vector 나 matrix 로 긴 과거를 저장하기에 충분하다고 보는 것은 지나친 단순화일 수 있다. (Q5) 긴 과거를 효과적으로 저장 · 기억하려면 deep memory 모듈이 필요한가?

기여와 로드맵 (Contributions and Roadmap)

이 논문은 위 다섯 질문에 답하기 위해, test time 에 기억을 효율적 · 효과적으로 학습하는 long-term neural memory module 을 설계하고, 이를 아키텍처에 통합하는 방법을 논한다.

Neural Memory (§ 3). 우리는 (meta in-context 모델로서) test time 에 데이터를 자신의 매개변수로 기억 · 저장하는 법을 학습하는 (deep) neural long-term memory 를 제시한다. 인간의 long-term memory 시스템에서 영감을 받아, 기대를 위배하는 (즉 놀라운) 사건이 더 잘 기억 되도록 설계한다. 이를 위해 입력의 놀라움 (surprise) 을 associative memory loss 에 대한 신경망의 gradient 로 측정한다 (§ 3.1). 제한된 메모리를 더 잘 다루기 위해, 메모리 크기와 데이터 놀라움의 비율을 고려하는 감쇠 (decay) 메커니즘을 도입한다. 우리는 이 감쇠 메커니즘이 사실 최

신 recurrent 모델의 forgetting 메커니즘을 일반화한 것임을 보인다. 흥미롭게도 이 메커니즘은 **momentum** 과 **weight decay** 를 동반한 **mini-batch gradient descent** 로 **meta 신경망을 최적화하는 것과 동치임**을 알 수 있다. mini-batch gradient descent 를 tensorize 해 matmul 연산을 더 쓰는 기법 위에서, 우리는 deep neural long-term memory 를 빠르게 병렬 학습하는 알고리즘을 제시한다.

Titans 아키텍처 (§ 4). long-term neural memory 를 설계한 뒤, 남은 문제는 이 메모리를 딥러닝 아키텍처에 어떻게 효과적으로 통합하느냐다. 우리는 세 개의 hyper-head 로 구성된 딥 모델 계열인 Titans 를 제시한다. (1) **Core**: short-term memory 로 이루어지며 데이터 처리의 주된 흐름을 담당한다 (제한된 window 크기의 attention 사용). (2) **Long-term Memory**: 긴 과거를 저장 · 기억하는 우리의 neural long-term memory 모듈. (3) **Persistent Memory**: 과제에 관한 지식을 부호화하는, 학습 가능하지만 데이터에 무관한 매개변수 집합. 개념 증명으로 우리는 메모리를 (i) context 로, (ii) layer 로, (iii) gated branch 로 통합하는 세 가지 Titans 변형을 제시한다.

실험 결과 (§ 5). language modeling, common-sense reasoning, recall 집약 과제, needle-in-haystack, time series forecasting, DNA modeling 에서 평가한다. Titans 는 포괄적 벤치마크에서 모든 최신 recurrent 모델과 그 hybrid 변형 (sliding-window attention 결합) 을 능가한다. 또한 같은 context window 의 Transformer 를 능가하고, 전체 context 를 쓰는 Transformer 와도 경쟁력 있는 성능을 보인다. 그러면서도 Transformer 와 달리 2M 이상의 context window 로 확장된다.

2 준비 (Preliminaries)

이 절에서는 논문 전반에서 쓰는 표기와 배경 개념을 논한다. 입력을 $x \in \mathbb{R}^{N \times d_{in}}$, neural memory 모듈을 $\mathcal{M}$, attention 의 query/key/value 를 Q, K, V , attention mask 를 $\mathbf{M}$ 으로 둔다. sequence 를 분할할 때 i 번째 segment 는 $S^{(i)}$ 로 쓴다. 아래첨자는 행렬 · 벡터 · segment 의 특정 원소를 가리키는 데 남용해 쓰며, 예컨대 $S_j^{(i)}$ 는 i 번째 segment 의 j 번째 토큰이다. 예외는 아래첨자 t 로, 이는 시간에 대한 recurrence 혹은 시각 t 에서의 신경망 상태를 가리키는 데 예약해 둔다. 신경망 $\mathcal{N}$ 과 데이터 표본 x 에 대해, $\mathcal{N}(x)$ 는 가중치조정을 **동반하는** forward pass 를, $\mathcal{N}^*(x)$ 는 가중치조정을 **동반하지 않는** forward pass 를 가리킨다. 또 $\mathcal{N}^{(k)}$ 는 신경망의 k 번째 layer 를 뜻한다.

2.1 배경 (Backgrounds)

Attention. 입력 $x \in \mathbb{R}^{N \times d_{in}}$ 이 주어지면, causal attention 은 입력에 의존하는 key/value/query 행렬에 대한 softmax 로 출력 $y \in \mathbb{R}^{N \times d_{in}}$ 을 계산한다.

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V \quad (1)$$

$$y_i = \frac{\sum_{j=1}^i \exp(Q_i^\top K_j / \sqrt{d_{in}}) V_j}{\sum_{\ell=1}^i \exp(Q_i^\top K_\ell / \sqrt{d_{in}})} \quad (2)$$

여기서 $W_Q, W_K, W_V \in \mathbb{R}^{d_{in} \times d_{in}}$ 은 학습가능한 매개변수다. 이 식은 각 query 가 앞선 모든 key 와의 유사도로 value 들을 가중 평균한다는 뜻이다. Transformer 는 출력을 계산하는 데 최소 $N \times d$ 연산이 필요해, 긴 sequence 에서 메모리 소비가 크고 throughput 이 낮다.

Efficient Attention. softmax attention 의 메모리 · throughput 을 개선하려는 연구는 I/O 를 고려한 구현, attention 행렬의 희소화, softmax 근사, kernel 기반 (linear) attention 등으로 나뉜다. 여기서는 linear attention 에 집중한다. 표준 attention 의 softmax 를 $\phi(x, y) = \phi(x)\phi(y)$ 를 만족하는 kernel 함수 $\phi(\cdot, \cdot)$ 로 대체하면 attention 은 다음과 같이 쓸 수 있다.

$$y_i = \frac{\phi(Q_i)^\top \sum_{j=1}^i \phi(K_j) V_j}{\phi(Q_i)^\top \sum_{\ell=1}^i \phi(K_\ell)} \quad (3)$$

이 식은 $\sum_j \phi(K_j)$ 와 $\sum_\ell \phi(K_\ell)$ 항이 매단계 재사용되므로 throughput 이 높아진다는 뜻이다. kernel 을 항등으로 택하면 위 식을 recurrent 형태로도 쓸 수 있다.

$$\mathcal{M}_t = \mathcal{M}_{t-1} + K_t^\top V_t, \quad (4)$$

$$y_t = Q_t \mathcal{M}_t \quad (5)$$

이는 linear attention 이 효율적 추론을 가능케 함을 보여 준다.

최신 linear 모델과 그 메모리 관점. 학습을 유용한 메모리를 획득하는 과정으로 정의하면, RNN 의 hidden state 는 모델이 정보를 압축해 넣는 메모리 단위로 볼 수 있다. 일반적 RNN 에서 hidden state 는 메모리 단위이고 recurrence 과정은 읽기 · 쓰기 연산으로 나뉜다. $x \in \mathbb{R}^{N \times d_{in}}$ 을 입력, $\mathcal{M} \in \mathbb{R}^d$ 를 메모리 단위, $y \in \mathbb{R}^{d_{in}}$ 을 출력이라 하면 일반적 RNN 은 다음과 같다.

$$\mathcal{M}_t = f(\mathcal{M}_{t-1}, x_t), \quad (\text{쓰기 연산}) \quad (6)$$

$$y_t = g(\mathcal{M}_t, x_t) \quad (\text{읽기 연산}) \quad (7)$$

이 관점에서 linear Transformer 의 recurrence(식 4) 는 key 와 value (K_t, V_t) 를 matrix 값 메모리 $\mathcal{M}_t$ 에 덧셈으로 압축 · 기록하는 것과 동치다. 이 덧셈적 성질 때문에 긴 context 에서는 메모리 오버플로가 일어나 성능이 크게 손상된다. 이를 해결하려는 두 방향이 있다. (1) **forget 메커니즘** 추가: GLA, LRU, Griffin, xLSTM, Mamba2 처럼 필요할 때 메모리를 지울 수 있는 적응적 (데이

터의존) forget gate 를 도입한다. (2) **쓰기 연산 개선**: 전통적 recurrent 모델의 덧셈적 쓰기를 극복하기 위해 Widrow 와 Hoff 는 Delta Rule 을 제시했는데, key-value 쌍을 더하기 전에 그 과거 value 를 먼저제거한다. DeltaNet 은 이를 sequence 길이에 대해 병렬화했고, Gated DeltaNet 은 여기에 forget gate 를 추가했다.

Memory 모듈. 메모리는 늘 신경망 설계의 핵심이었다. 선형 layer 를 key-value(associative) 메모리 시스템으로 보는 발상은 fast weight program 으로 거슬러 올라가며, 여기서 동적 fast program 이 RNN 에 쓰기가능한 메모리로 통합된다. Hebbian 규칙과 delta 규칙이 가장 널리 쓰인학습 규칙이다. 다만 이런 모델들은 모두 **momentary surprise(순간적놀라움)**에 기반해 sequence 의 토큰 흐름 (token flow) 을 놓치고, 대부분 forget gate 가 없어 메모리 관리가 부실하다. 최신 모델과 우리 아키텍처의연결은 부록 C 에서, 추가 관련 연구는 부록 A 에서 논한다.

3 Test Time 에 기억하기를 학습하기 (Learning to Memorize at Test Time)

long-term memory 의 부재를 극복하고 모델이 정보를학습 · 망각 · 검색하도록, 이 절에서는 test time 에기억하기를 학습하는 meta 모델인 neural long-term memory module 을제시한다. § 3.1 에서 동기와 설계를, § 3.2 에서 빠른 병렬 학습을, § 3.3 에서 persistent memory 증강을 다룬다.

3.1 Long-term Memory

long-term memory 모듈을 설계하려면 과거 이력의 추상을 자신의 매개변수로부호화할 수 있는 모델이 필요하다. LLM 이 학습 데이터를 기억한다는 사실이그 예다. 그러나 memorization 은 대개 바람직하지 않은 현상으로 알려져왔다. 일반화를 제한하고, 프라이버시 우려를 낳으며, test time 성능을떨어뜨린다. 게다가 test 데이터가 분포 밖 (out-of-distribution) 일 수있으므로 학습 데이터를 외운 것이 test time 에 도움이 되지 않을 수 있다. 우리는 **test time 에 데이터를 기억 · 망각하는 법을학습하는 online meta-model** 이 필요하다고 주장한다. 이 설정에서 모델은 memorization 이 가능한 함수를 학습하되 학습 데이터에 과적합하지 않아, test time 에 더 나은 일반화를 얻는다.

학습 과정과 surprise 지표. long-term memory 학습의 핵심 발상은이를 online learning 문제로 다루어, 과거 정보 $x_1, \dots, x_{t-1}$ 을 long-term neural memory 모듈 $\mathcal{M}_t$ 의 매개변수로 압축하는것이다. 기대를 위배하는 (놀라운) 사건이 더 잘 기억된다는 인간 기억의특성에서 영감을 받아, 모델의 놀라움을 **입력에 대한 gradient** 로단순하게 정의한다. gradient 가 클수록 입력이 과거 데이터와 더 다르다는뜻이다. 이 surprise 점수로 메모리를 다음과 같이 갱신한다.

$$\mathcal{M}_t = \mathcal{M}_{t-1} - \theta_t \underbrace{\nabla \ell(\mathcal{M}_{t-1}; x_t)}_{\text{Surprise}}$$

(8)

이 식은 현재 토큰이 저장된 내용을 얼마나 위배하는지를 gradient 로 재고, 그만큼 메모리를 갱신한다는 뜻이다. 그러나 이 surprise 지표만으로는 큰놀라움 직후에 오는 중요한 정보를 놓칠 수 있다. 여러 번 놀라운 단계를거치면 gradient 가 극단적으로 작아져 평평한 영역 (국소 최소) 에 갇히고 sequence 의 일부 정보를 놓치기 때문이다. 인간 기억의 관점에서, 어떤사건은 기억할 만해도 긴 시간 내내 우리를 놀라게 하지는 않는다. 처음순간이 충분히 놀라워 긴 시간대 내내 주의를 끌고, 그 시간대 전체를기억하게 만드는 것이다. 이를 개선하기 위해 surprise 를 (1) 아주 최근과거의 놀라움인 **past surprise** 와 (2) 새로 들어오는 데이터의놀라움인 **momentary surprise** 로 나눈다.

$$\mathcal{M}_t = \mathcal{M}_{t-1} + S_t, \quad (9)$$

$$S_t = \eta_t \underbrace{S_{t-1}}_{\text{Past Surprise}} - \theta_t \underbrace{\nabla \ell(\mathcal{M}_{t-1}; x_t)}_{\text{Momentary Surprise}} \quad (10)$$

이 식은 **momentum** 을 동반한 **gradient descent** 와 유사하며, S_t 가 momentum 항이다. 즉 momentum 은 시간 (sequence 길이) 에 걸친 **놀라움의 기억** 역할을 한다. η_t 는 놀라움이 시간에 따라 어떻게 감쇠하는지를 제어하는 데이터 의존 surprise decay (x_t 의 함수) 이고, θ_t 는 momentary surprise 를 최종 surprise 지표에 얼마나 반영할지를 데이터 의존적으로 제어한다. 이 데이터 의존성이 특히 중요하다. 이전 토큰의 놀라움이 다음 토큰의 놀라움에 영향을 주는 것은 대개 모든 토큰이 서로 관련되어 같은 context 에 있을 때만 타당하기 때문이다. 따라서 데이터 의존적 η 는 (1) $\eta_t \rightarrow 0$ 으로 지난 놀라움을 무시하거나 (context 전환 가능성), (2) $\eta_t \rightarrow 1$ 로 지난 놀라움을 온전히 반영할지 (최근 토큰과 강하게 관련) 를 제어할 수 있다.

목적함수 (Objective). 위 surprise 지표는 손실 함수 $\ell(\cdot; \cdot)$ 에 기반하며, 이것이 메모리가 test time 에 학습해 수행하는 목적이다. 즉 메모리는 손실 ℓ 에 따라 함수를 학습하는 meta 모델이다. 이 연구에서는 과거 데이터를 key-value 쌍으로 저장하는 **associative memory** 에 집중한다. x_t 가 주어지면 두 선형 layer 로 key 와 value 를 사영한다.

$$k_t = x_t W_K, \quad v_t = x_t W_V, \quad (11)$$

여기서 $W_K, W_V \in \mathbb{R}^{d_{in} \times d_{in}}$ 이다. 메모리가 key 와 value 의 연관을 학습하도록 손실을 다음과 같이 정의한다.

$$\ell(\mathcal{M}_{t-1}; x_t) = \|\mathcal{M}_{t-1}(k_t) - v_t\|_2^2 \quad (12)$$

이 손실을 meta 모델 (메모리) 의 **inner-loop** 에서 최적화함으로써, 모델은 test time 에 key 와 value 의 매핑을 기억하는 법을 학습한다. meta-learning 모델처럼 메모리의 학습은 inner-loop 에서 이루어지므로 W_K, W_V 는 위 손실의 **하이퍼파라미터** 다. 즉 inner-loop 에서는 $\mathcal{M}$ 의 가중치를, **outer-loop** 에서는 아키텍처의 나머지 매개변수를 최적화한다.

Forgetting 메커니즘. 수백만 토큰 같은 아주 긴 sequence 를 다룰 때는, 깊거나 아주 큰 matrix 값 메모리라도 어떤 과거 정보를 잊어야 할지 관리하는 것이 결정적이다. 이를 위해 더 이상 필요 없는 정보를 잊게 하는 적응적 forgetting 메커니즘을 도입해 갱신 규칙을 수정한다.

$$\mathcal{M}_t = (1 - \alpha_t) \mathcal{M}_{t-1} + S_t, \quad (13)$$

$$S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t), \quad (14)$$

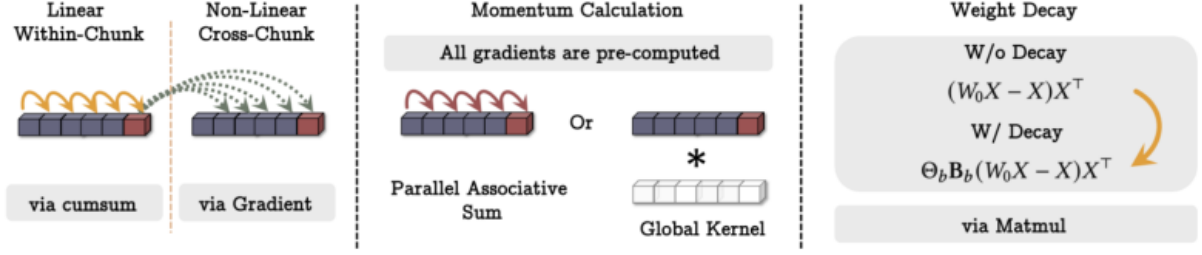


그림 1: neural memory 의 학습이 어떻게 병렬로, matmul 을 써서 이루어질 수 있는지에 대한 도식. 출처: 원저자, arXiv:2501.00663 Fig.1

여기서 $\alpha_t \in [0, 1]$ 은 메모리를 유연하게 제어하는 gating 이다. $\alpha_t \rightarrow 0$ 이면 과거 추상을 건드리지 않고 메모리를 갱신하고, $\alpha_t \rightarrow 1$ 이면 메모리 전체를 지운다. 이 weight decay 메커니즘은 최신 RNN 의 gating 메커니즘과 밀접하게 관련된다. 정리하면, 전체 메모리쓰기는 세 개의 스칼라가 토큰마다 gating 되는 **momentum·weight decay** 를 동반한 **mini-batch gradient descent** 한 스텝이다.

Memory 아키텍처. 이 논문에서는 $L_M \geq 1$ 개 layer 의 단순 MLP 를 long-term memory 의 아키텍처로 삼는다. 그 이유는 long-term memory 의 설계 동기와 통합 방식을 명확히 하는 데 집중하기 위함이다. vector 값이나 matrix 값 메모리를 쓰면 메모리 모듈은 과거데이터를 하나의 선 (line) 에 압축·적합시킨다. 즉 meta-learning/online-learning 관점에서 matrix 값 메모리 $\mathcal{M} = W \in \mathbb{R}^{d_{in} \times d_{in}}$ 을 쓰는 것은 $\ell(W_{t-1}; x_t) = \|W_{t-1}k_t - v_t\|_2^2$ 를 최적화하는 것과 동치인데, 이는 **online linear regression** 목적이므로 과거데이터의 의존 관계가 선형이라고 가정한다. 반면 우리는 **deep memory 모듈** ($L_M \geq 2$) 을 주장한다. 2 개 이상 layer 의 MLP 가 선형 모델보다 엄밀히 더 표현력 있다는 이론적 결과에 부합하며, § 5.5 에서 deep memory 가 실제로도 더 효과적임을 보인다.

Memory 검색 (Retrieving a Memory). 메모리로부터 어떻게 정보를 되찾는가? 우리는 가중치 갱신 없는 forward pass(추론) 로 query 에 해당하는 메모리를 검색한다. 입력 x_t 에 선형 layer W_Q 를 적용해 $q_t = x_t W_Q$ 를 얻고, 다음으로 해당 정보를 되찾는다.

$$y_t = \mathcal{M}^*(q_t) \quad (15)$$

이 식은 읽기가 메모리 가중치를 바꾸지 않는 순수 추론임을 뜻한다.

3.2 Long-term Memory 학습을 어떻게 병렬화할 것인가

앞에서 보았듯, long-term memory 모듈의 설계는 associative memory 손실 $\ell(\mathcal{M}_{t-1}; x_t) = \|\mathcal{M}_{t-1}(k_t) - v_t\|_2^2$ 를 momentum·weight decay 를 동반한 gradient descent 로 최적화하는 meta 모델 학습과 동치다. 따라서 이론상 long-term memory 학습에는 $O(N)$ FLOP 이 든다 (N 은 sequence 길이). 그러나 실제로는 학습을 병렬화하고 하드웨어 가속기 (TPU, GPU) 를 온전히 활용하기 위해 과정을 tensorize 하고 matmul 을 더 써야 한다.

이제 mini-batch gradient descent, 데이터 의존 learning rate, weight decay 로 inner-loop 가중치를 계산하는 과정이 matmul 과 sum 만으로 재구성됨을 보인다. mini-batch gradient descent(상수 learning rate) 로 최적화하는 모델의 forward pass 가 matmul 로 계산될 수 있다

는 결과 위에서, sequence 를 크기 $b \geq 1$ 의 chunk 로 나누어 mini-batch gradient descent 를 다음과 같이 쓴다.

$$\mathcal{M}_t = (1 - \alpha_t)\mathcal{M}_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t) = \beta_t \mathcal{M}_0 - \sum_{i=1}^t \theta_i \frac{\beta_t}{\beta_i} \nabla \ell(\mathcal{M}_{t'}; x_i), \quad (16)$$

여기서 $t' = t - \text{mod}(t, b)$ 이고 $\beta_i = \prod_{j=1}^i (1 - \alpha_j)$ 이다. 이 식은 chunk 안의 모든 gradient 를 **chunk 시작 시점의 가중치** $\mathcal{M}_{t'}$ 에서 취한다는 뜻이다 (순수 online GD 대신 mini-batch). 단순화를 위해 첫 chunk ($t = b, t' = 0$) 와 선형 메모리 $\mathcal{M}_t = W_t$ 경우를 보면, 손실을 이용해 다음을 얻는다.

$$\nabla \ell(W_0; x_t) = (W_0 x_t - x_t) x_t^\top \Rightarrow \sum_{i=1}^b \theta_i \frac{\beta_b}{\beta_i} \nabla \ell(W_0; x_i) = \Theta_b \mathbf{B}_b (W_0 X - X) X^\top, \quad (17)$$

여기서 $\Theta_b = \text{diag}(\theta_1, \theta_2, \dots, \theta_b)$ 이고 $\mathbf{B}_b$ 는 β_b/β_i 들로 유사하게 정의된 대각행렬이다. 이 식은 chunk 안 모든 gradient 가 같은 고정 가중치를 쓰므로 하나의 batched matmul 로 동시에 계산됨을 뜻한다. 모든 $\Theta_b^k, \mathbf{B}_b^k$ 를 한꺼번에 저장할 필요 없이 chunk 별로만 저장하므로 메모리를 덜 쓴다. MLP 메모리 ($L_{\mathcal{M}} \geq 2$) 의 경우도 유사하다.

momentum 항의 병렬화. momentum 항을 살펴보면,

$$S_t = \eta_t S_{t-1} - \theta_t u_t, \quad (18)$$

여기서 $u_t = \nabla \ell(\mathcal{M}_{t'}; x_t)$ 이다. 모든 u_t 는 동시에 계산할 수 있으므로 식 (18) 은 u_t 를 입력, S_t 를 hidden state, η_t 를 입력 의존 전이값으로 갖는 **1 차 선형 recurrence** 다. 따라서 **parallel associative scan** 으로 chunk 안 모든 S_t 를 계산할 수 있다.

매개변수를 chunk 의 함수로. $\alpha_t, \theta_t, \eta_t$ 를 토큰 x_t 의 함수 대신 **chunk 의 함수** 로 만들 수도 있다. 표현력은 잃지만 학습을 더 빠르게 할 수 있다. 이 경우 각 chunk 안에서 α, θ, η 가 같은 값을 쓰므로, 식 (17) 에서 Θ 를 단일 스칼라로 저장할 수 있다. η, θ 가 chunk 안에서 학습가능해지 시불변이면 식 (18) 은 **선형 시불변 (LTI) 시스템** 이 되어 global convolution 으로 계산된다. 우리 실험에서는 이 매개변수를 토큰의 함수로 두지만, 이런 단순화는 더 큰 모델을 효율적으로 학습하는 향후 연구대상이 될 수 있다.

3.3 Persistent Memory

우리의 long-term memory 는 출력이 전적으로 context 에 의존하므로 **contextual memory** 로도 볼 수 있다. 그래서 우리는 long-term memory 외에, 과제 관련 메모리 역할을 하는 학습 가능하지만 입력에 무관한 매개변수 집합도 함께 쓴다. 이런 메모리는 문헌에서 persistent 혹은 meta-memory 로 불린다. $N_p \geq 1$ 에 대해 학습 가능한 매개변수 $P = [p_1 \ p_2 \ \dots \ p_{N_p}]$ 를 sequence 앞에 붙인다.

$$x_{new} = [p_1 \ p_2 \ \dots \ p_{N_p}] \parallel x, \quad (19)$$

여기서 $\parallel$ 는 연결 (concatenation) 이다. persistent memory 의 동기를 세관점에서 논한다.

메모리 관점. neural long-term memory 는 모든 매개변수가 입력의존적인 contextual memory 다. 효과적 메모리 시스템은 과제 지식의 추상을 저장할 **입력 무관 매개변수**도 필요로 한다. 과제를 익힌다는 것은 그 과제를 어떻게 수행하는지에 대한 지식을 기억하는 일이며, 이 매개변수가 그 지식을 담당한다.

Feedforward Network 관점. Transformer 의 attention 뒤 fully-connected layer 는 데이터 무관 매개변수를 가진 attention 가중치와 유사함이 알려져 있다. Sukhbaatar 등은 fully-connected layer 의 ReLU 를 Softmax 로 바꾸면 attention 과 비슷한, 데이터 무관 가중치가 나옴을 보였다.

$$FFN(x) = W_V \text{Softmax}(W_K x) \quad (20)$$

여기서 W_K, W_V 는 입력 무관일 때 attention 의 K, V 행렬처럼 작동한다. persistent memory 가중치도 같은 기능을 기대하는데, 이를 sequence 앞부분에 두면 입력 무관 attention 가중치를 갖게 된다.

기술적 관점. causal mask 를 쓴 attention 은 sequence 초기 토큰에 대한 암묵적 편향이 있어 초기 토큰에 거의 항상 높은 attention 가중치를 부여하고, 이는 성능을 손상시킨다. sequence 앞의 학습 가능한 매개변수는 attention 가중치를 더 효과적으로 재분배해 이런 효과 (attention sink) 를 완화한다.

4 메모리를 어떻게 통합할 것인가 (How to Incorporate Memory?)

남은 중요한 질문은 설계한 neural memory 를 딥러닝 아키텍처에 어떻게 효과적 · 효율적으로 통합하느냐다. 메모리 관점에서 Transformer 의 K, V 행렬 쌍은 associative memory 블록으로 해석되며, 정확한 의존 관계 모델링과 제한된 context window 때문에 **short-term memory** 로 본다. 반면 데이터로부터 계속 학습해 가중치에 저장하는 neural memory 는 **long-term memory** 역할을 할 수 있다. 이 절에서 세 가지 Titans 변형을 제안하며, 각 변형은 아주 긴 context 에서 효율성과 효과성 사이의 절충을 보인다.

4.1 Memory as a Context (MAC)

첫 설계에서는 메모리를 현재 정보에 대한 **context** 로 다룬다. 긴 sequence $x \in \mathbb{R}^{N \times d_{in}}$ 을 먼저 고정 크기 segment $S^{(i)} (i = 1, \dots, N/C)$ 로 나눈다. 들어오는 segment $S^{(t)}$ 를 현재 context 로, 그 이전 segment 들을 과거 정보로 본다. segment $S^{(t)}$ 직전의 long-term memory 상태를 $\mathcal{M}_{t-1}$ 이라 하면, 입력 context 를 query 로 삼아 메모리에서 해당 정보를 되찾는다.

$$h_t = \mathcal{M}_{t-1}^*(q_t), \quad (21)$$

여기서 $q_t = S^{(t)} W_Q$ 이다. 이어 이 과거 정보와 persistent memory 를 attention 모듈의 입력 sequence 로 쓴다.

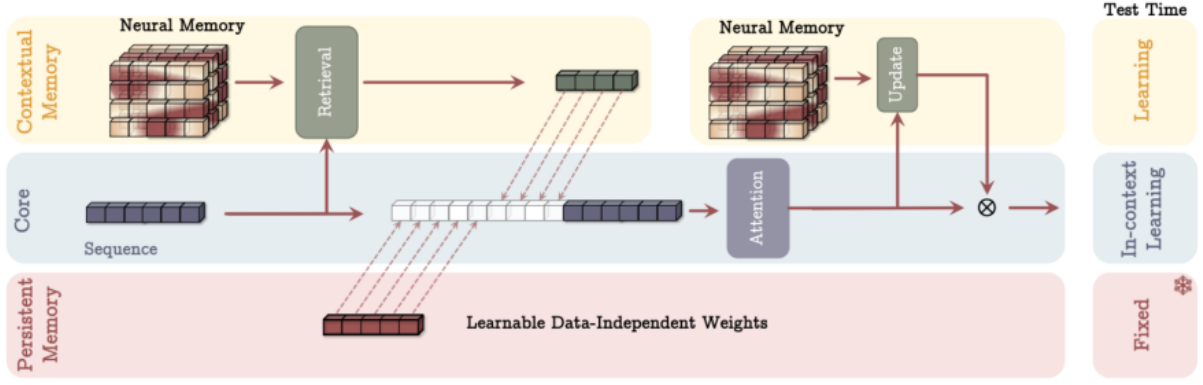


그림 2: Memory as a Context (MAC) 아키텍처. (1) core, (2) contextual(long-term) memory, (3) persistent memory 의 세 branch 로 이루어진다. core branch 는 해당 long-term·persistent memory 를 입력 sequence 와 연결한다. 이어 attention 이 sequence 에 수행되어 어떤 정보를 long-term memory 에 저장할지 결정한다. test time 에 contextual memory 에 해당하는 매개변수는 여전히 학습 중이고, core branch 매개변수는 in-context learning 을 담당하며, persistent memory 매개변수는 과제 지식을 저장하므로 고정된다. 출처: 원저자, arXiv: 2501.00663 Fig.2

$$\tilde{S}^{(t)} = [p_1 \ p_2 \ \dots \ p_{N_p}] \parallel h_t \parallel S^{(t)}, \quad (22)$$

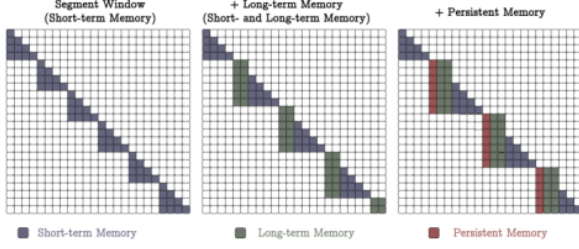
$$y_t = \text{Attn}(\tilde{S}^{(t)}) \quad (23)$$

전체 sequence 에 대한 attention 맵 구조는 그림 3a 에 나온다. 그 다음 y_t 로 다음 segment 를 위한 long-term memory 를 갱신하고 최종 출력을 만든다.

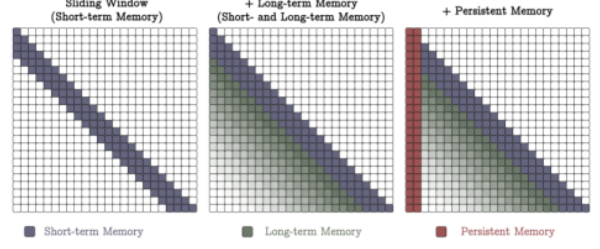
$$\mathcal{M}_t = \mathcal{M}_{t-1}(y_t), \quad (24)$$

$$o_t = y_t \otimes \mathcal{M}_t^*(y_t) \quad (25)$$

즉 $\mathcal{M}_{t-1}$ 의 가중치를 forward pass 를 통해 갱신한다. 이설계의 이점은 다음과 같다. (1) attention 이 과거와 현재 context 를 모두가지므로, 현재 데이터에 대해 long-term memory 정보가 필요한지 판단할 수 있다. (2) attention 은 long-term memory 가 현재 context 에서 유용



(a) **Memory as a Context (MAC)**. We segment the sequence and use full causal attention in each window. Again, the first N_p tokens are persistent memory and the next N_l are long-term memory tokens



(b) **Memory as Gating (MAG)**. We use sliding window attention (SWA) as a short-term memory and our neural memory module as a long-term memory, combining by a gating.

그림 3: Titans 변형별 attention mask. (a) Memory as a Context (MAC): sequence 를 분할하고 각 window 에서 full causal attention 을 쓴다. 앞 N_p 개는 persistent memory, 다음 N_l 개는 long-term memory 토큰이다. (b) Memory as Gating (MAG): sliding-window attention(SWA) 을 short-term memory 로, neural memory 모듈을 long-term memory 로 쓰고 gating 으로 결합한다. 출처: 원저자, arXiv:2501.00663 Fig.3

한정보만 저장하도록 돕는다. 각 segment 의 모든 토큰이 유용하지는 않으며전부 외우면 메모리 오버플로가 날 수 있으므로, attention 이 어떤 정보가유용한지 이해하도록 도와 메모리 용량을 더 잘 관리한다. (3) test time: (i) persistent memory 매개변수는 과제 지식을 부호화하므로 고정 되고, (ii) attention 모듈 가중치는 in-context learner 이며, (iii) long-term memory 모듈은 test time 에도 여전히 학습 (기억) 한다. 즉 가중치가 긴 과거의 추상을부호화하므로 test time 에도 neural memory 의 가중치를 갱신한다.

4.2 Gated Memory (MAG)

다음 변형에서는 한 branch 에서 입력 데이터로 직접 long-term memory 를갱신하고, 다른 branch 에서 sliding-window attention(SWA) 을 쓴다.

$$\tilde{x} = [p_1 \ p_2 \ \dots \ p_{N_p}] \parallel x, \quad (26)$$

$$y = \text{SW-Attn}^*(\tilde{x}), \quad (27)$$

$$o = y \otimes \mathcal{M}(\tilde{x}), \quad (28)$$

여기서 SW-Attn^* 는 prefix 를 둔 sliding-window attention 이다 (그림 3b). 앞 설계와 달리 입력 데이터를 분할하지 않는다. $\mathcal{M}(\tilde{x})$ 는 sequence 토큰들에 대한 모든 recursion 을마친 뒤 메모리의 최종 출력을 가리킨다. $\otimes$ 는 임의의 비선형 gating 일 수 있으며, 실험에서는 y 와 $\mathcal{M}(\tilde{x})$ 를 학습가능한 vector 값 가중치로 정규화하고 비선형 $\sigma(\cdot)$ 을 적용한뒤 결합한다. 이 설계에서 sliding-window attention 은 정밀한 short-term memory 로, neural memory 모듈은

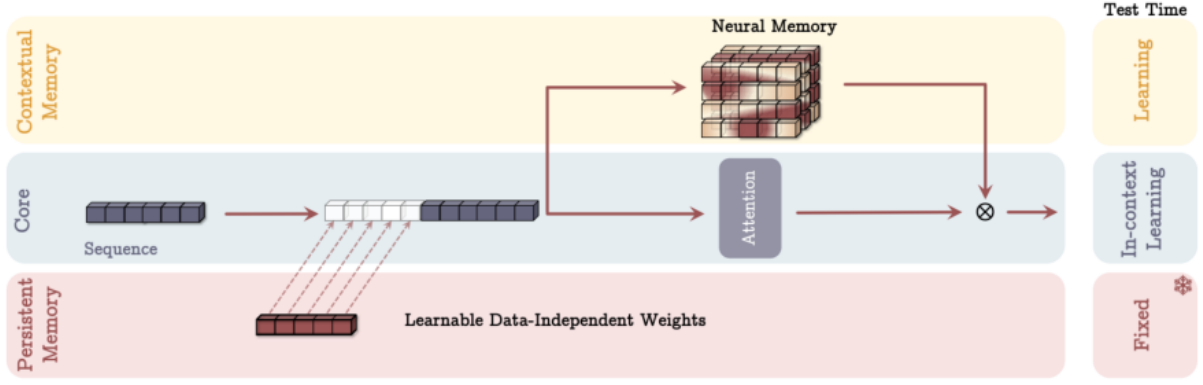


그림 4: Memory as a Gate (MAG) 아키텍처. 마찬가지로 (1) core, (2) contextual memory, (3) persistent memory 의 세 branch 를 갖는다. 다만 persistent memory 만 context 에 통합하고, 메모리는 gating 으로 core branch 와 결합한다. test time 동작은 그림 2 와 같다. 출처: 원저자, arXiv:2501.00663 Fig.4

사그라드는 (fading) long-term memory 로 작동한다. 이는 구조가 서로 다른 head 들을 가진 multi-head 아키텍처로도 볼수 있다.

4.3 Memory as a Layer (MAL)

마지막 변형은 neural memory 를 딥 신경망의 한 **layer** 로 쓴다. 이설계는 문헌에서 더 흔한 형태로, hybrid 모델이 recurrent 모델을 full 또는 sliding-window attention 과 쌓는 방식이다. 입력 x 에 대해,

$$\tilde{x} = [p_1 \ p_2 \ \dots \ p_{N_p}] \parallel x, \quad (29)$$

$$y = \mathcal{M}(\tilde{x}), \quad (30)$$

$$o = \text{SW-Attn}(y), \quad (31)$$

여기서 SW-Attn 은 sliding-window attention 이다. 이 설계의 주된단점은 모델의 성능이 각 layer 에 의해 제약되어 attention 과 neural memory 의 상보적 데이터 처리를 활용하지 못한다는 점이다. 실험에서는 H3 와 유사한 아키텍처를 쓰되 sequence 모델을 neural memory 모듈 (LMM) 로 대체한다.

Attention 없는 메모리. 위에서 MAL 을 LMM 과 attention 의 순차결합으로 논했지만, 단순 변형으로 LMM 을 attention 없는 sequence 모델로 다룰 수도 있다. 메모리 시스템의 각 부분

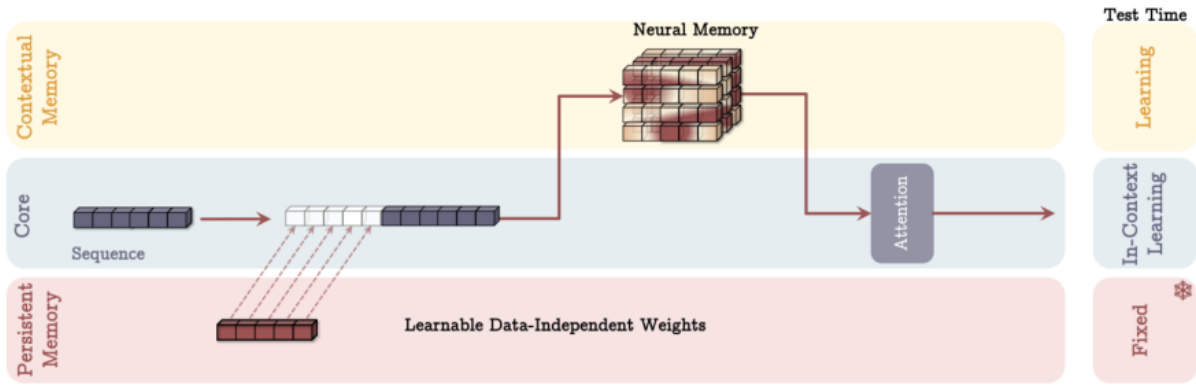


그림 5: Memory as a Layer (MAL) 아키텍처. 여기서 memory layer 는 attention 모듈에 앞서 과거와 현재 context 를 압축하는 역할을 한다. 출처: 원저자, arXiv:2501.00663 Fig.5

은 다른 요소가 교란되어도 독립적으로 작동해야 하므로, long-term memory 모듈은 short-term memory(attention) 없이도 강력한 모델이어야 한다. 이 변형을 실험에서 **LMM** 또는 **Titans (LMM)** 으로 부른다.

4.4 아키텍처 세부사항

간결한 서술을 위해 residual connection, 선형 layer gating, 정규화 같은 구현 세부는 본문에서 생략했다. 모든 블록에서 residual connection 을 쓴다. query/key/value 계산에는 비선형 활성화 함수로 SiLU($\cdot$) 를 쓰고, query 와 key 는 ℓ_2 -norm 으로 정규화한다.

Convolution. 최신 linear recurrent 모델을 따라, query·key·value 사영 뒤에 각각 1D depthwise-separable convolution layer 를 넣는다. 성능에 큰 영향은 없으나 개선을 보였고 계산적으로도 효율적이다.

Gating. 최종 출력 사영 전에 정규화와 선형 layer gating 을 쓴다.

정리 4.1. Transformer, 대각 linear recurrent 모델, DeltaNet 은 모두 TC^0 에 제한되지만, Titans 는 TC^0 를 넘어서는 문제를 풀 수 있다. 즉 state tracking 과제에서 Titans 는 Transformer 와 대부분의 최신 linear recurrent 모델보다 이론적으로 더 표현력이 크다.

5 실험 (Experiments)

Titans 와 그 변형을 language modeling, common-sense reasoning, needle-in-haystack, DNA modeling, time series forecasting 에서 평가한다. 다음 실증 질문에 답한다. (1) downstream 과제에서 Titans 는 baseline 대비 어떠한가?(§ 5.2, § 5.6, § 5.7) (2) Titans 의 실제 context 길이는?(§ 5.3, § 5.4) (3) context 길이에 대해 어떻게 확장되는가?(§ 5.8) (4) 메모리 깊이가 성능·효율에 미치는 영향은?(§ 5.5) (5) 각 구성 요소의 기여는?(§ 5.9)

참고: v1 에서는 Titans 의 학습 패러다임이 왜 효과적인지에 대한 통찰·근거를 제공하는 데 초점을 둔다. 더 큰 모델의 결과는 마무리 중이며 다음 버전에서 보고할 예정이다.

5.1 실험 설정

모델. 세 가지 Titans 변형 — (1) Memory as a Context (MAC), (2) Memory as a Gate (MAG), (3) Memory as a Layer (MAL) — 과 (4) neural memory 모듈 단독 (LMM) 에 집중한다. 각 모델에 대해 (i) 170M, (ii) 340M, (iii) 400M, (iv) 760M 의 네 규모를 둔다. 앞 셋은 FineWeb-Edu 에서 뽑은 15B 토큰으로, 마지막 (760M) 은 같은 데이터셋의 30B 토큰으로 학습한다.

Baseline. language 과제에서 Transformer++, RetNet, GLA, Mamba, Mamba2, DeltaNet, TTT, Gated DeltaNet 과 비교한다. needle-in-haystack 에서는 GPT-4, Llama3+RAG, RecurrentGemma2-9B, Mistral 등과도 비교한다. time series 에서는 Mamba 기반, Transformer 기반, linear 모델과 비교한다.

학습. Gated DeltaNet 의 학습 절차를 따라 Llama 2 tokenizer(어휘 32K) 와 학습 길이 4K 토큰을 쓴다. AdamW 옵티마이저 (learning rate $4e-4$, cosine annealing), 배치 0.5M 토큰, weight decay 0.1 을 쓴다.

5.2 Language Modeling

language modeling perplexity 와 common-sense reasoning 을 먼저 본다. 340M/400M/760M 규모의 결과는 표 1 에 있다. 비-hybrid 모델 중 (Transformer++ 포함) 우리 neural memory 모듈이 perplexity 와 정확도 모두에서 최고 성능을 낸다. 같은 gradient 기반 recurrent 모델인 TTT 와의 비교는 우리 weight decay 와 momentum 의 중요성을 보여 준다. weight decay 는 필요할 때 과거를 잊는 gating 으로 해석되고, momentum 은 surprise 지표에 추가 메모리를 제공해 메모리 관리를 돕는다. Mamba, Mamba2, Gated DeltaNet 같은 일부 baseline 도 gating 을 쓰지만, 우리 neural memory 의 우월성은 우리 surprise 메커니즘과 **deep·비선형 메모리**의 중요성을 보여 준다.

hybrid 모델에서는 세 Titans 변형 (MAC, MAG, MAL) 모두 Samba(Mamba + attention) 와 Gated DeltaNet-H2 를 능가한다. Titans (MAL) 의 우월성은 아키텍처와 attention 이 동일하므로 neural memory 모듈의 힘 덕분으로 볼 수 있다. MAG 와 MAC 는 성능이 서로 가깝지만, MAC 가 데이터의 더 긴 의존 관계를 다룰 때 더 낫다. 흥미롭게도 MAG 와 MAC 모두 MAL 변형을 능가한다. 이는 현재 문헌의 hybrid 모델 (Hymba 제외) 이 MAL 방식의 결합을 쓰고 있기에 특히 중요한 발견이다.

표 1: language modeling 과 common-sense reasoning 에서 Titans 와 recurrent·Transformer 기반 baseline 의 성능. hybrid 모델은 *로 표시. Wiki/LMB 은 perplexity(↓), 나머지는 정확도 (↑).

Model	Wiki ppl ↓	LMB ppl ↓	LMB acc ↑	PIQA	Hella	Wino	ARC- e	ARC- c	SIQA	BoolQ	Avg ↑
340M params / 15B to- kens											
Trans- former++	31.52	41.08	30.76	62.98	34.76	50.53	45.21	24.05	36.81	58.24	42.92
Ret- Net	32.50	49.73	28.24	62.61	34.15	50.91	44.27	23.62	36.79	59.72	42.54
GLA	28.51	43.02	28.73	64.05	35.96	50.00	54.19	24.29	37.13	58.39	44.09
Mamba	30.83	40.21	29.94	63.79	35.88	49.82	49.24	24.56	35.41	60.07	43.59
DeltaNet	28.65	47.30	28.43	63.52	35.95	49.63	52.68	25.37	37.96	58.79	44.04
TTT	27.44	34.19	30.06	63.97	35.71	50.08	53.01	26.11	37.32	59.83	44.51
Gated DeltaNet	27.01	30.94	34.11	63.08	38.12	51.60	55.28	26.77	34.89	59.54	45.42
Ti- tans (LMM)	26.18	29.97	34.98	64.73	39.61	51.85	55.60	28.14	34.52	59.99	46.17
Ti- tans (MAC)*	25.43	28.13	36.00	65.32	40.35	51.21	58.17	29.00	38.63	60.18	47.36

Model	Wiki ppl ↓	LMB ppl ↓	LMB acc ↑	PIQA	Hella	Wino	ARC- e	ARC- c	SIQA	BoolQ	Avg ↑
Ti- tans (MAG)*	25.07	28.72	36.71	64.88	40.56	52.49	57.72	28.16	39.75	60.01	47.54
Ti- tans (MAL)*	24.69	28.80	35.74	64.97	39.44	51.97	56.58	28.21	38.14	57.32	46.55
400M params / 15B to- kens											
Trans- former++	30.63	37.37	29.64	64.27	37.72	51.53	54.95	27.36	38.07	61.59	45.64
Ret- Net	29.92	46.83	29.16	65.23	36.97	51.85	56.01	27.55	37.30	59.66	45.47
HGRN2	32.33	47.14	26.12	64.52	35.45	52.24	55.97	25.51	37.35	59.02	44.52
GLA	27.96	36.66	27.86	65.94	37.41	49.56	56.01	26.36	38.94	59.84	45.24
Mamba	29.22	39.88	29.82	65.72	37.93	50.11	58.37	26.70	37.76	61.13	45.94
Mamba2	26.34	33.19	32.03	65.77	39.73	52.48	59.00	27.64	37.92	60.72	46.91
DeltaNet	27.69	44.04	29.96	64.52	37.03	50.82	56.77	27.13	38.22	60.09	45.57
TTT	26.11	31.52	33.25	65.70	39.11	51.68	58.04	28.99	38.26	59.87	46.86
Gated DeltaNet	25.47	29.24	34.40	65.94	40.46	51.46	59.80	28.58	37.43	60.03	47.26
Samba*	25.32	29.47	36.86	66.09	39.24	51.45	60.12	27.20	38.68	58.22	47.23
Gated DeltaNet- H2*	24.19	28.09	36.77	66.43	40.79	52.17	59.55	29.09	39.04	58.56	47.69
Ti- tans (LMM)	25.03	28.99	35.21	65.85	40.91	52.19	59.97	29.20	38.74	60.85	47.83
Ti- tans (MAC)*	25.61	27.73	36.92	66.39	41.18	52.80	60.24	29.69	40.07	61.93	48.65
Ti- tans (MAG)*	23.59	27.81	37.24	66.80	40.92	53.21	60.01	29.45	39.91	61.28	48.60
Ti- tans (MAL)*	23.93	27.89	36.84	66.29	40.74	52.26	59.85	29.71	38.92	58.40	47.87
760M params / 30B to- kens											
Trans- former++	25.21	27.64	35.78	66.92	42.19	51.95	60.38	32.46	39.51	60.37	48.69
Ret- Net	26.08	24.45	34.51	67.19	41.63	52.09	63.17	32.78	38.36	57.92	48.46
Mamba	28.12	23.96	32.80	66.04	39.15	52.38	61.49	30.34	37.96	57.62	47.22
Mamba2	22.94	28.37	33.54	67.90	42.71	49.77	63.48	31.09	40.06	58.15	48.34
DeltaNet	24.37	24.60	37.06	66.93	41.98	50.65	64.87	31.39	39.88	59.02	48.97
TTT	24.17	23.51	34.74	67.25	43.92	50.99	64.53	33.81	40.16	59.58	47.32
Gated DeltaNet	21.18	22.09	35.54	68.01	44.95	50.73	66.87	33.09	39.21	59.14	49.69

Model	Wiki ppl ↓	LMB ppl ↓	LMB acc ↑	PIQA	Hella	Wino	ARC- e	ARC- c	SIQA	BoolQ	Avg ↑
Samba*	20.63	22.71	39.72	69.19	47.35	52.01	66.92	33.20	38.98	61.24	51.08
Gated	19.88	20.83	39.18	68.95	48.22	52.57	67.01	35.49	39.39	61.11	51.49
DeltaNet- H2*											
Ti- tans (LMM)	20.04	21.96	37.40	69.28	48.46	52.27	66.31	35.84	40.13	62.76	51.56
Ti- tans (MAC)	19.93	20.12	39.62	70.46	49.01	53.18	67.86	36.01	41.87	62.05	52.51
Ti- tans (MAG)	18.61	19.86	40.98	70.25	48.94	52.89	68.23	36.19	40.38	62.11	52.50
Ti- tans (MAL)	19.07	20.33	40.05	69.99	48.82	53.02	67.54	35.65	30.98	61.72	50.97

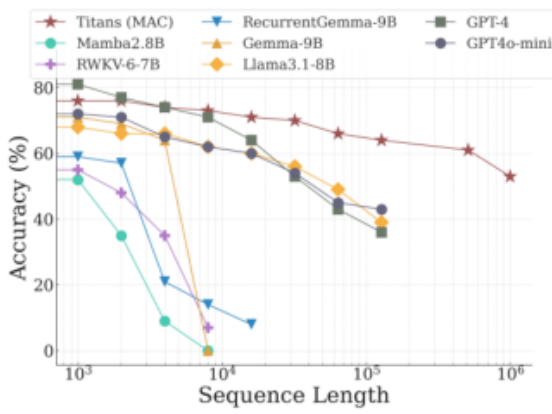
5.3 Needle in a Haystack

모델을 더 긴 context window 로 확장한다고 해서 아주 긴 sequence 에서 효과적이라는 뜻은 아니다. needle-in-a-haystack(NIAH) 과제는 모델의 실제 유효 context 길이를 재도록 설계되었다. 긴 방해 텍스트 (“haystack”) 속에서 한 조각 정보 (“needle”) 를 되찾는 능력을 본다. RULER 벤치마크의 Single NIAH(S-NIAH) 를 써서 길이 2K/4K/8K/16K 에서 평가한다 (표 2). neural memory 모듈이 세 과제 모두에서 최고 성능을 낸다. 세 가지 핵심 차이 덕분이다. (1) TTT 대비, momentum 과 forgetting(weight decay) 으로 메모리 용량을 더 잘 관리하므로 길이가 늘어도 성능이 떨어지지 않고 일관된 추세를 보인다. (2) Mamba2 대비, deep 비선형 메모리를 가져 메모리 관리가 낫다. 또 Mamba2 는 메모리를 제거할 수 없어 길이가 늘면 성능이 크게 떨어진다. (3) DeltaNet 대비, DeltaNet 은 delta rule 로 메모리를 대체할 수는 있으나 지울 (erase) 수는 없어 forgetting 이 없다. Titans 변형들은 동등하거나 더 나은 결과를 내며, 최고는 MAC 다.

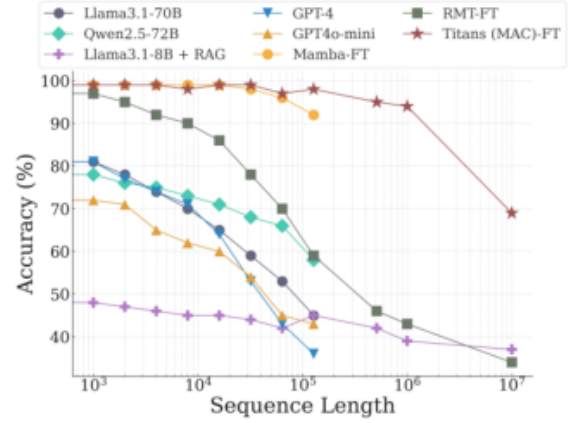
표 2: RULER 벤치마크의 S-NIAH 과제에서 Titans 와 baseline 의 성능.

Model	PK- 2K	PK- 4K	PK- 8K	PK- 16K	N- 2K	N- 4K	N- 8K	N- 16K	W- 2K	W- 4K	W- 8K	W- 16K
TTT	98.4	98.8	98.0	88.4	60.2	36.6	10.2	4.4	78.8	28.0	4.4	0.0
Mamba2	98.6	61.4	31.0	5.4	98.4	55.8	14.2	0.0	42.2	4.2	0.0	0.0
DeltaNet	96.8	98.8	98.6	71.4	47.2	15.4	12.8	5.4	46.2	20.0	1.6	0.0
Ti- tans (LMM)	99.8	98.4	98.2	96.2	100.0	99.8	93.4	80.2	90.4	89.4	85.8	80.6
Ti- tans (MAC)	99.2	98.8	99.0	98.4	99.6	98.2	97.6	97.4	98.2	98.2	95.6	95.2
Ti- tans (MAG)	99.4	98.0	97.4	97.4	99.2	98.8	97.2	98.6	98.0	98.0	90.2	88.2
Ti- tans (MAL)	98.8	98.6	98.8	97.8	99.8	98.1	96.8	96.4	98.0	97.4	92.0	90.4

Model	S-NIAH-PK				S-NIAH-N				S-NIAH-W			
	2K	4K	8K	16K	2K	4K	8K	16K	2K	4K	8K	16K
TTT	98.4	98.8	98.0	88.4	60.2	36.6	10.2	4.4	78.8	28.0	4.4	0.0
Mamba2	98.6	61.4	31.0	5.4	98.4	55.8	14.2	0.0	42.2	4.2	0.0	0.0
DeltaNet	96.8	98.8	98.6	71.4	47.2	15.4	12.8	5.4	46.2	20.0	1.6	0.0
Titans (LMM)	99.8	98.4	98.2	96.2	100.0	99.8	93.4	80.2	90.4	89.4	85.8	80.6
Titans (MAC)	99.2	98.8	99.0	98.4	99.6	98.2	97.6	97.4	98.2	98.2	95.6	95.2
Titans (MAG)	99.4	98.0	97.4	97.4	99.2	98.8	97.2	98.6	98.0	98.0	90.2	88.2
Titans (MAL)	98.8	98.6	98.8	97.8	99.8	98.1	96.8	96.4	98.0	97.4	92.0	90.4



(a) Few-shot Setup



(b) Fine-Tuning Setup

그림 6: BABILong 벤치마크에서 Titans와 baseline의 성능. Titans (MAC)는 GPT-4 같은 극대 모델을 포함해 모든 baseline을 능가한다. (a) few-shot 설정, (b) fine-tuning 설정. 출처: 원저자, arXiv:2501.00663 Fig.6

5.4 BABILong 벤치마크

앞 절은 needle 하나만 되찾는 단순 NIAH였다. 아주 긴 sequence에서의 진짜이점을 보기 위해, 이 절에서는 극단적으로 긴 문서에 흩어진 사실들을 가로질러 추론해야 하는 더 어려운 BABILong 벤치마크를 쓴다. 원 실험설정과 학습 절차를 따르며 두 설정을 둔다. (1) 대규모 사전학습 모델을 쓰는 **few-shot** 설정, (2) MAC 변형을 fine-tune해 다른 fine-tune baseline과 비교하는 **fine-tuning** 설정.

few-shot 결과 (그림 6a)에서 Titans는 모든 baseline — Mamba-2.8B, RWKV-6-7B, RecurrentGemma-9B, Gemma-9B, Llama3.1-8B, GPT-4, GPT4o-mini — 을 능가한다. 이는 Titans (MAC)가 baseline보다 훨씬 적은 매개변수를 가지면서 이룬 결과다.

fine-tuning 설정에서는 작은 Titans (MAC)를 (i) 거의 같은 규모의 작은 fine-tune 모델 (Mamba, RMT), (ii) RAG를 붙인 대형 모델 (Llama3.1-8B), (iii) 극단적으로 큰 모델 (GPT-4, GPT4o-mini, Qwen2.5-72B, Llama3.1-70B)과 비교한다 (그림 6b). Titans는 GPT-4 같은 극대 모델조차 능가한다. RMT 같은 메모리 부착 Transformer 대비 더 나은데, RMT는 과거를 16 크기 vector 값 메모리로 압축하는 반면 Titans는 in-context online 메모리 학습기로 과거를 매개변수에 부호화하기 때문이다. 흥미롭게도 Llama3.1-8B에 RAG를 붙여도 약 70배 적은 매개변수의 Titans보다 성능이 낮다.

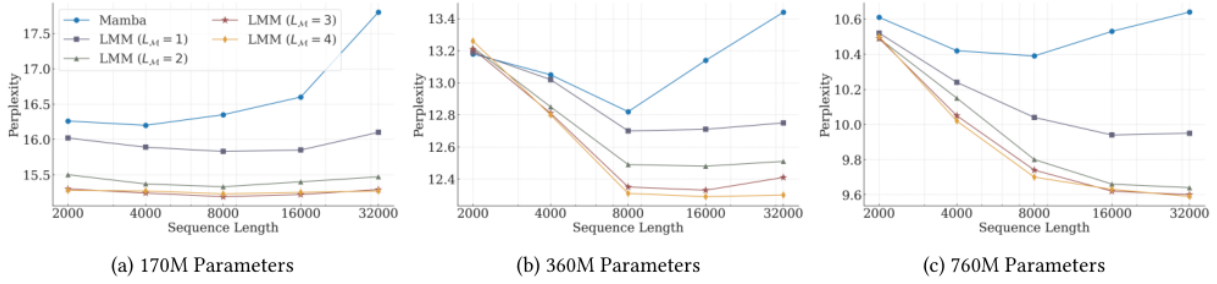


그림 7: 메모리 깊이가 perplexity 에 미치는 효과. 더 깊은 long-term memory 는 긴 sequence 에서 더 나은 스케일링을 낸다. (a) 170M, (b) 360M, (c) 760M 매개변수. 출처: 원저자, arXiv: 2501.00663 Fig.7

5.5 Deep Memory 의효과

deep memory 가 wall-clock 학습 시간과 성능에 미치는 영향을 평가한다. neural memory 모듈의 $L_M = 1, 2, 3, 4$ 변형에 집중하고, Mamba 를 성능 baseline 으로 쓴다. 공정한 비교를 위해 모두 같은 학습 절차로 Pile 데이터셋 부분집합에서 학습한다.

sequence 길이의 함수로 perplexity 를 보면 (그림 7), 메모리 깊이 L_M 이 커질수록 모든 sequence 길이에서 더 나은 perplexity 를 얻는다. 또 깊은 메모리는 매개변수 수가 적을 때 sequence 길이에 더 강건하다. 매개변수가 늘면 모든 모델이 긴 sequence 에서 더 나은성능을 보인다.

메모리 깊이가 학습 throughput 에 미치는 영향도 본다 (그림 8). 모든 모델이 context 길이에 대해 선형으로 확장된다 (초당 토큰 수가 sequence 길이에대해 상수 추세). 다만 메모리 깊이가 커지면 예상대로 학습이 선형적으로느려진다. 따라서 깊은 메모리가 늘 효율적인 것은 아니며, 효과성과 효율성사이의 절충이 존재한다.

5.6 Time Series Forecasting

메모리 모듈의 효과를 더 넓은 과제에서 보기 위해 time series forecasting 도 평가한다. Simba 프레임워크를 쓰되 그 Mamba 모듈을 우리 neural memory 로 대체한다. ETT, ECL, Traffic, Weather 벤치마크에서 평가한결과 (표 3), 우리 neural memory 모듈이 Mamba 기반 ·linear 기반 ·Transformer 기반 아키텍처를 포함한 모든 baseline 을능가한다.

표 3(발췌): long-term forecasting 에서 Neural Memory 모듈의 성능 (MSE/MAE, 낮을수록 좋음). 원문 표 3 은 Simba·PatchTST·iTransformer·RLinear·Crossformer·TiDE·TimesNet·DLinear 를함께 비교하며, Neural Memory 가 전 데이터셋에서 최고다. 예: ETTm1 에서 Neural Memory 0.358/0.387 대 PatchTST 0.387/0.400.

데이터셋	Neural Memory MSE	Neural Memory MAE
ETTM1	0.358	0.387
ETTM2	0.261	0.309
ETTTh1	0.350	0.401
ETTTh2	0.336	0.382
ECL	0.162	0.261
Traffic	0.415	0.289
Weather	0.231	0.265

5.7 DNA Modeling

자연어를 넘어선 능력을 이해하기 위해 DNA modeling 도 평가한다. GenomicsBenchmarks 의 downstream 과제에서 사전학습 모델을 평가한다. Titans (LMM) 의 결과 (표 4) 를 보면, LMM

	Neural Memory		Simba		iTransformer		RLinear		PatchTST		Crossformer		TiDE		TimesNet		DLinear	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETtm1	0.358	0.387	0.383	0.396	0.407	0.410	0.414	0.407	0.387	0.400	0.513	0.496	0.419	0.419	0.400	0.406	0.403	0.407
ETtm2	0.261	0.309	0.271	0.327	0.288	0.332	0.286	0.327	0.281	0.326	0.757	0.610	0.358	0.404	0.291	0.333	0.350	0.401
ETTh1	0.420	0.421	0.441	0.432	0.454	0.447	0.446	0.434	0.469	0.454	0.529	0.522	0.541	0.507	0.458	0.450	0.456	0.452
ETTh2	0.336	0.382	0.361	0.391	0.383	0.407	0.374	0.398	0.387	0.407	0.942	0.684	0.611	0.550	0.414	0.427	0.559	0.515
ECL	0.162	0.261	0.169	0.274	0.178	0.270	0.219	0.298	0.205	0.290	0.244	0.334	0.251	0.344	0.192	0.295	0.212	0.300
Traffic	0.415	0.289	0.493	0.291	0.428	0.282	0.626	0.378	0.481	0.304	0.550	0.304	0.760	0.473	0.620	0.336	0.625	0.383
Weather	0.231	0.265	0.255	0.280	0.258	0.278	0.272	0.291	0.259	0.281	0.259	0.315	0.271	0.320	0.259	0.287	0.265	0.317

Using Llama3.1-8B model with RAG performs worse than Titans with about $\times 70$ less parameters.

The Effect of Deep Memory

In this section, we evaluate the effect of deep memory in both wall-clock training time and model performance². To this end, we focus on different variants of our neural memory module, where $L_M = 1, 2, 3, 4$. We also use Mamba as a baseline to compare model performance. For a fair comparison, we use the same training process for all models and train them on a subset of the Pile dataset (L. Gao et al. 2020).

We report the perplexity of our models and baselines as the function of the sequence length in Figure 7. Interestingly, with the increase of memory depth, L_M , the model can achieve better perplexity over all sequence length. Also, deeper memory models are more robust to the sequence length when the model has less number of parameters. With the increase of the number of parameters, all models show better performance on longer sequences.

To evaluate the effect of memory depth ($L_M = 1, 2, 3, 4$) on the training throughput, we report the training throughput (the number of tokens per second) as the function of sequence length in Figure 8. All models scale linearly with respect to the context length (i.e., constant trend in the number of tokens per second with respect to sequence length). Also, by increasing the memory depth as expected, we can see a linear trend that a deeper memory results in higher training throughput. Therefore, it is not always efficient to use deeper memory models, showing a trade-off between effectiveness and efficiency.

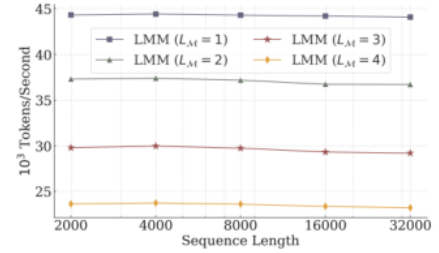


그림 8: 메모리 깊이가 학습 throughput 에 미치는 효과. 출처: 원저자, arXiv:2501.00663 Fig.8

은 여러 genomics downstream 과제에서 최첨단 아키텍처와 경쟁력 있다.

표 4: GenomicsBenchmarks 에서 사전학습 DNA 모델의 downstream 평가. top-1 분류 정확도 (%). (‘—’ 는 원문 미보고)

Model	Enhancer Cohn	Enhancer Ens	Human Reg.	Non-TATA Promoters	Human OCR Ens.
CNN	69.5	68.9	93.3	84.6	68.0
DNABERT	74.0	85.7	88.1	85.6	75.1
GPT	70.5	83.5	91.5	87.7	73.0
HyenaDNA	74.2	89.2	93.8	96.6	80.9
Trans-former++	73.4	89.5	89.9	94.4	79.5
Mamba	73.0	89.5	89.5	96.6	79.0
Based	74.6	—	—	96.8	—
Neural Memory Module	75.2	89.6	89.3	96.6	79.9

5.8 효율성 (Efficiency)

neural memory 와 Titans 의 효율성을 최첨단 sequence 모델과 비교한다. 여러 (sequence 길이 $\times$ 배치 크기) 에서 학습 throughput 을 본다 (그림 9). recurrent 모델을 비교하면, 우리 메모리 모듈은 Mamba2-Gated DeltaNet 보다 약간 느린데, 주된 이유는 (1) deep memory 와 더 표현력 있는전이 과정 (메모리 갱신), (2) Mamba2 구현의 고도로 최적화된 kernel 때문이다. 흥미롭게도 Titans (MAL) 은 baseline 과 메모리 모듈보다 빠르는데, 이는 SWA 와 full attention 구현에 쓰인 FlashAttention 의 고도로 최적화된 kernel 덕분이다.

5.9 Ablation 연구

끝으로 Titans 의 다양한 설계 선택에 대해 ablation 을 수행한다. neural memory 모듈을 기본 모델로 삼고 한 번에 한 요소씩 바꾼다. (1) deep memory 를 linear memory 로 교체, (2) convolution 제거, (3) surprise 의 momentum 제거, (4) weight decay(forget) 제거, (5) persistent memory 제거. 결과 (표 5) 를 보면 모든 요소가 성능에 긍정적으로 기여하며, 기여크기는 weight decay > momentum > convolution > persistent memory 순이다.

아키텍처 설계의 효과. 세 대표 변형을 (i) language modeling, (ii) common-sense reasoning, (iii) long-context NIAH(BABILong) 에서 비교한다. MAC 와 MAG 는 language modeling-reasoning 에서 성능이 가깝지만, long-context NIAH 에서는 MAC 가 유의미하게 낮다. 둘 다 MAL 보다 낮다. 그림 9 와 함께 보면, 이는 빠른 학습과 더 표현력 있는 설계사이의 절충을 보여준다.

표 5: Titans 의 ablation. 모든 요소가 성능에 긍정적으로기여한다.

Model	Language Modeling ppl ↓	Reasoning acc ↑	Long Context acc ↑
LMM	27.01	47.83	92.68
+Attn (MAC)	26.67	48.65	97.95
+Attn (MAG)	25.70	48.60	96.70
+Attn (MAL)	25.91	47.87	96.91
Linear Memory	28.49	46.97	85.34
w/o Convolution	28.73	45.82	90.28
w/o Momentum	28.98	45.49	87.12
w/o Weight Decay	29.04	45.11	85.60
w/o Persistent Memory	27.63	46.35	92.49

HyenaDNA	74.2	89.2	93.8	96.6	80.9
Transformer++	73.4	89.5	89.9	94.4	79.5
Mamba	73.0	-	-	96.6	-
Based	74.6	89.5	89.5	96.8	79.0
Neural Memory Module	75.2	89.6	89.3	96.6	79.9

mba module with our neural memory. We report the results on common time series forecasting benchmark ECL, Traffic, and Weather (H. Zhou et al. 2021). The results are reported in Table 3. Our neural memory outperforming all baselines, including Mamba-based, linear-based, and Transformer-based architectures.

Modeling

To understand the capability of Titans beyond natural language, we further evaluate the performance of our module on DNA modeling tasks. To this end, we evaluate pre-trained models on the downstream tasks benchmarks (Grešová et al. 2023). We follow the same experimental setups from Nguyen et al. (2024), and report results of baselines by Arora et al. (2024). The performance of Titans (LMM) and baselines are reported in Table 4. We find that LMM is competitive with state-of-the-art architectures across different downstream genomics

Efficiency

To compare the efficiency of our neural memory as well as Titans with state-of-the-art sequence models. The training throughput of models for different sequence length $\times$ batch size are reported in Figure 9. Comparing with baselines, including our neural memory module, we can see our memory module is slightly slower than Mamba2 and Gated DeltaNet, mainly due to: (1) the more expressive transition process (memory update), and (2) the lack of an optimized kernel in the implementation of Mamba2. Interestingly, Titans are faster than baselines as well as the memory module. The reason for this better throughput is the highly optimized kernel of FlashAttention-2 (Arora et al. 2024), which is used for implementing SWA and full attention

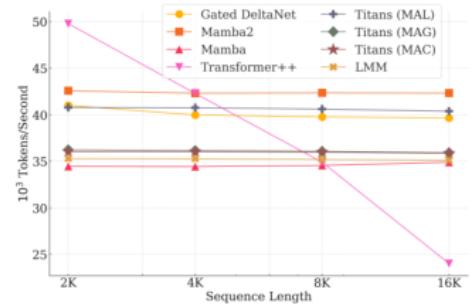


그림 9: Titans 와 baseline 의 학습 throughput 비교. 출처: 원저자, arXiv:2501.00663 Fig.9

6 결론 (Conclusion)

이 논문은 meta in-context learner 로서 test time 에 기억하기를 학습하는 neural long-term memory 를 제시했다. 이 메모리 모듈은 본질적으로 recurrent 모델이며, 더 놀랍거나 놀라운 토큰에 가까운 토큰들을 적응적으로 기억한다. 최신 recurrent 모델과 비교해 더 표현력 있는 메모리 갱신·저장 메커니즘을 갖는다. 이 메모리를 써서 우리는 Titans 아키텍처와 세 변형 — 메모리를 (1) context 로, (2) gating 으로, (3) layer 로 통합 — 을 제시했다. 다양한 과제에 대한 실험은 Titans 가 특히 긴 context 에서 Transformer 와 최신 linear recurrent 모델보다 더 효과적임을 확인해 준다. 즉 Titans 는 2M 이상의 context window 로 확장되면서도 baseline 보다 나은 정확도를 낸다. Titans 는 PyTorch 와 JAX 로 구현되었으며, 학습·평가에 쓴 코드는 곧 공개할 예정이다.

부록 A: 관련 연구 (Related Work)

Titans 혹은 그 구성 요소로 독립적으로 이어질 수 있는 여러 관점이 있다. 세 범주로 나누어 살펴본다.

A.1 Linear Recurrent 모델

Transformer 의 학습·추론 비용을 줄이기 위해 linear recurrent 모델이 큰 주목을 받았다. **1 세대** (RetNet, LRU, RWKV, S5, S4) 는 데이터 무관 전이 행렬/감쇠 메커니즘을 쓴다. **2 세대**는 전통적 RNN 의 gating 을 이런 linear 아키텍처에도 도입했다 (Griffin, SSM 계열, RWKV6). **3 세대**는 meta-learning, online learning, delta-rule 에 기반한 더 복잡한 메모리 갱신 규칙을 써서 더 표현력 있는 모델을 낸다 (Longhorn, Gated DeltaNet, TTT, DeltaNet). 우리 LMM 은 토큰 흐름 (token flow) 을 메모리 갱신에 통합한, 이런 모델들의 다음 세대로 볼 수 있다.

A.2 Transformer 기반 아키텍처

Transformer 는 attention 기반의 사실상 표준 backbone 이나 제공비용으로 긴 context 확장이 어렵다. 이를 개선하려는 연구는 I/O 인지 구현, attention 행렬 희소화, softmax 근사, kernel 기반 (linear) attention 으로 나뉜다.

Segment 기반 Transformer. 또 다른 방향은 segment/chunk 기반 Transformer 다. chunk Transformer 의 단점은 segment 가 완전히 분리되어 context window 가 chunk 길이로 제한된다는 것이다. 이를 해결하려 여러 연구가 chunk 간 정보 전달을 돕는 메모리의 중요성을 논했다. Titans 와의 핵심 차이는 (1) 이런 모델의 메모리는 단순한 작은 vector 라 복잡한 정보 압축에 표현력이 부족하고, (2) forget 메커니즘이 없어 빠른 오버플로가 나며, (3) momentary surprise 에만 집중해 정보 흐름을 놓친다는 점이다. 특히 Recurrent Memory Transformer (RMT) 에 견주면, Titans (MAC) 는 vector 값 소형 메모리 대신 neural memory 모듈을 쓰는 RMT 의 일반화로 볼 수 있다.

LLM 을 위한 메모리. 학습 후 LLM 에 외부 메모리 모듈을 붙이는 방향도 있다. 우리 접근과 달리 이들은 메모리를 초기 아키텍처의 일부로 두어 end-to-end 학습하지 않으며, 대부분 chunk 기반 Transformer 와 같은 한계를 겪는다.

A.3 Test Time Training 과 Fast Weight Program

메모리 설계와 증강. 문헌에서는 지식 추상을 기억하는 메모리 (persistent memory) 나, recurrence/Transformer/gradient 등으로 데이터의 존 정보 (contextual memory) 를 기억하는 모듈을 설계하려는 노력이 많았다. 그러나 이들은 (1) momentary surprise 기반이라 데이터 흐름·사건을 놓치거나, (2) forget 메커니즘이 없어 빠른 오버플로가 나거나, (3) 고정 크기의 얇은 (matrix 값) 메모리라 긴 context 에서 성능이 부실하거나, (4) test time 에 매개변수가 고정되어 적응이 없다.

Fast Weight Program. 선형 layer를 key-value 메모리로 보는발상은 fast weight program으로 거슬러 오르며 Hebbian·delta 규칙이 널리 쓰였다. 그러나 이들 모두 momentary surprise 기반이라 토큰흐름을 놓치고 대부분 forget gate가 없어 메모리 관리가 부실하다.

Test Time Training. test time에 학습하기/learning to learn의 핵심 발상은 local learning의 초기 연구 (각 test 표본을 예측전에 이웃으로 학습)로 거슬러 오른다. 우리와 가장 유사한 연구는 MNM과 TTT-layer이며, 그 핵심 차이는 부록 C에서 논한다.

부록 C: Sequence 모델로서의 LMM (LMM as a Sequence Model)

LMM이 sequence 모델로서 최신 linear recurrent 모델과 어떻게 연결되는지 논한다. 단순화를 위해 선형 메모리 $\mathcal{M}_t = W_t \in \mathbb{R}^{d_{in} \times d_{in}}$ 부터 시작한다. 이 경우 목적함수는 $\ell(\mathcal{M}; x_t) = \frac{1}{2} \|\mathcal{M}_t k_t - v_t\|_2^2$ 가 되고, momentum·weight decay를 동반한 gradient descent로 최적화한다. 식 (13)의 recurrence를 다시 보면,

$$\mathcal{M}_t = \text{diag}(1 - \alpha_t) \mathcal{M}_{t-1} + S_t \quad (32)$$

$$S_t = \text{diag}(\eta_t) S_{t-1} - \text{diag}(\theta_t) (\mathcal{M}_{t-1} k_t^\top k_t - v_t^\top k_t) \quad (33)$$

LMM은 Gated DeltaNet의 일반화다. DeltaNet은 $\mathcal{L} = \frac{1}{2} \|S_t k_t - v_t\|_2^2$ 를 최적화하는 online learning 문제로 해석될 수 있고, 이는 다음을 낳는다.

$$S_{t+1} = S_t - \theta_t \nabla \mathcal{L} = S_t (I - \theta_t k_t k_t^\top) + \theta_t v_t k_t^\top \quad (34)$$

Gated DeltaNet은 여기에 weight decay 항을 더한 것이다. 식 (32)와 (34)를 비교하면 $\eta_t = 0$ 으로 두면 두 식이 동치가 된다. 따라서 LMM은 Gated DeltaNet을 세 측면에서 일반화한다.

- **Momentum 기반 규칙:** Delta Rule은 momentary surprise 기반이라 토큰 흐름이 메모리 갱신에 영향을 주지 못한다. LMM은 momentum 규칙으로 과거와 순간의 놀라움을 모두 고려한다.
- **Deep Memory:** Gated DeltaNet은 닫힌 recurrence 형태가 필요해 선형 (matrix 값) 메모리에 제한되지만, LMM은 gradient 기반 정식화로 deep memory를 허용해 표현력이 더 크다.
- **비선형 recurrence:** DeltaNet/Gated DeltaNet은 선형 recurrence이지만, LMM은 inter-chunk 비선형 + intra-chunk 선형 recurrence를 써서 표현력이 더 크다.

RWKV-7 등도 같은 정식화와 손실을 쓰므로 LMM은 이런 모델들을 모두 일반화한다.

LMM은 Longhorn의 일반화다. Longhorn은 같은 손실을 쓰되 implicit online learning으로 닫힌 형태를 유도한다.

$$S_{t+1} = S_t (I - \delta_t k_t k_t^\top) + \delta_t v_t k_t^\top, \quad (35)$$

여기서 $\delta_t = \frac{\theta_t}{1+\theta_t k_t^\top k_t}$ 이다. 그러나 forget gate 가 없어 오버플로가 더 빠르다. 따라서 (1) momentum 규칙, (2) deep memory, (3) 비선형 recurrence 에 더해, LMM 은 (4) **forget gate** 의 이점을 가져 메모리 관리가 낫다.

LMM 은 TTT layer 의 일반화다. 우리가 아는 한 TTT 는 gradient 기반 갱신 규칙을 가진 유일한 최신 linear recurrent 모델이다. LMM 은 TTT 와 세가지 핵심 차이가 있다. (1) **Forgetting 메커니즘**: TTT 는 매 시점메모리를 갱신하되 과거를 잊을 기회가 없어, 메모리 크기를 고정하면 긴 sequence 를 관리하지 못한다. LMM 의 forget 메커니즘은 아주 오래된 정보가 더 필요 없을 때 메모리를 비운다. 이 forget 이 일반적으로 weight decay 와동치이며 병렬 학습에 빠르게 통합됨을 보였다. (2) **Momentum 기반 갱신 규칙**: TTT 는 momentary surprise 기반이지만 LMM 은 momentum 규칙으로 과거와 순간의 놀라움을 모두 고려한다. (3) **Deep Memory**: TTT-layer 도 더 깊은 메모리를 허용하지만 그 장단점을 실험적으로 평가한적이 없다.

우리가 아는 한, 우리 neural long-term memory 모듈은 **momentum 기반 갱신 규칙을 가진 최초의 linear recurrent 모델**이다. 끝으로, Griffin·DeltaNet·Gated DeltaNet·H3·Mamba2·Samba 등 최신 linear 모델의 hybrid 변형은 모두 순차적 layer 단위 설계에 기반한다. Titans 는 이런 메모리 모듈을 아키텍처에 얼마나 효과적으로 통합할 수있는지를 보인다.